Networking & SSH

Section 10 - Detailed Command Reference | nezuko homelab

Viewing Network Information

`ip a / ip addr`

Display all network interfaces and their IP addresses

What it does:
ip a (short for 'ip address') shows every network interface on the system and the IP addresses assigned to each one - both IPv4 (inet) and IPv6 (inet6). This is the modern replacement for the older ifconfig command.

Interface naming conventions on nezuko:

Prefix	Meaning	Example on nezuko
enp...	Ethernet interface	enp60s0 = 192.168.1.187
wlp...	WiFi interface (wireless)	wlp0s20f3 = 192.168.1.188
lo	Loopback - always present, represents 'this machine'	127.0.0.1
tailscale0	Tailscale VPN virtual interface	100.76.26.45
docker0	Docker's internal bridge network	172.17.0.1
veth...	Virtual ethernet - one end of a Docker container's network cable	[illegible]6944d

```
ip a # Show all interfaces and IPs on nezuko
ip a show enp60s0 # Show only the ethernet interface
ip route # Show the routing table
```

Concept: Ports

If an IP address is a building's street address, a **port** is one of the doors into that building. Without ports, a server could only run one service at a time - ports let many services run simultaneously on the same IP, each listening on its own numbered door. Ports are numbered 0 to 65535.

Why 65535?

Port numbers are stored as a 16-bit number. 2^16 = 65536 possible values, numbered 0 through 65535. This was simply the space allocated when TCP/IP was designed.

Range	Name	Properties	Examples
0-1023	Well-Known Ports	Reserved for standard system services. Require sudo/root to bind. Identical across every OS wo[illegible]	22=SSH, 53=DNS, 80=HTTP, 443=HTTPS
1024-49151	Registered Ports	Assigned to specific applications by convention. Don't need root. Can be changed in config.	8080=alt HTTP, 8096=Jellyfin, 9000=Portai[illegible]
49152-65535	Ephemeral / Dynamic Ports	Assigned randomly by the OS for OUTGOING connections. Not tied to any service. Created and [illegible]	Random - e.g. 41641, 35608

WARNING: Well-Known Ports (0-1023) are the SAME on every operating system - Windows, Linux, macOS all use port 22 for SSH. This is an IANA global standard. A sysadmin CAN deliberately run a service on a non-standard port (e.g. SSH on 2222) for security, but the standard itself never changes.

Checking what's listening on nezuko:

```
ss -tuln # Show all listening TCP/UDP ports
# -t = TCP -u = UDP -l = listening only -n = show numbers not names

sudo ss -tulnp # Add -p to show which process owns each port
```

ADDED: ss vs netstat

Added because: netstat is the older command you'll see referenced in older guides and some Linux+ material - ss is its modern, faster replacement on current systems.

netstat -tuln # Old command - same purpose as ss -tuln
ss -tuln # Modern equivalent - faster, preferred

Both show the same information; ss reads directly from the kernel and is faster on systems with many connections. Know both names for Linux+ but use ss day to day.

Concept: IPv4 vs IPv6

	IPv4 (inet)	IPv6 (inet6)
Created	1983	Created in response to IPv4 running out
Format	4 numbers, dot separated. 192.168.1.187	8 groups of hex, colon separated. fe80::6e02:e0ff:fed3:b944
Address space	approx 4.3 billion (2^32)	approx 340 undecillion = 3.4x10^38 (2^128)
Status	Still primary on most home networks	Running alongside IPv4 (dual stack)

The :: in an IPv6 address is shorthand for a run of zeros - IPv6 addresses are long, so this compresses them for readability. Most home networks and devices, including nezuko's Tailscale IP (100.76.26.45), primarily use IPv4 day to day, but every interface also has an inet6 address assigned automatically.

Commands: hostname / hostnamectl

`hostname / hostnamectl`

View and change the system's hostname

What they do:
hostname simply prints the current hostname. hostnamectl is the modern systemd tool that shows detailed system information (kernel, OS, architecture, virtualisation) AND can be used to change the hostname permanently.

Common options:

Command	What it does
hostname	Print current hostname only
hostnamectl	Show full system info - hostname, OS, kernel, architecture
sudo hostnamectl set-hostname NAME	Permanently change the hostname
hostnamectl set-hostname NAME --pretty	Set a 'pretty' display name with spaces/capitals

```
hostname # Shows: Nezuko
hostnamectl # Full system info including hostname
sudo hostnamectl set-hostname nezuko # Change hostname to 'nezuko'
```

Important File: /etc/hosts

File path:
/etc/hosts - a local, manual lookup table mapping hostnames to IP addresses.

Why it matters:
When you change the hostname with hostnamectl, /etc/hosts is NOT updated automatically. It still contains the OLD hostname next to the loopback address (127.0.1.1). If left unchanged, some local services and the shell prompt may behave inconsistently.

Format:
Each line is: IP address, then one or more hostnames, space separated.

```
127.0.0.1 localhost
127.0.1.1 oldname # <- this line needs updating after a hostname change
```

How to update it after changing hostname:

```
sudo nano /etc/hosts # or: sudo vi /etc/hosts
# Find the line: 127.0.1.1 oldname
# Change 'oldname' to your new hostname
# Save and exit (vi): Esc then :wq

hostnamectl # Verify - new hostname should now be shown
```

Concept: SSH (Secure Shell)

SSH creates an **encrypted connection** between two computers - a client (your Windows PC) and a server (nezuko) - over **port 22**. Everything typed and returned travels through this encrypted tunnel, so even on an untrusted network nobody can read your session.

How an SSH connection is established - step by step:

Step	What happens
1	You run: ssh user@server (e.g. ssh scottp@nezuko)
2	The client (Windows PC) connects to the server (nezuko) on port 22
3	Client and server negotiate encryption - a key exchange takes place
4	You authenticate - either Password Authentication or Key Authentication
5	An encrypted channel is established
6	Your commands now run on the remote server through that channel

---

Password Authentication vs Key Authentication

	Password Authentication	Key Authentication
Simplicity	Simple to set up	Requires one-time setup
Security	Less secure	More secure
Day to day use	Type password every time	No password needed once set up
Basis	Something you know	Cryptographic key pair - something you have
Vulnerability	Vulnerable to brute force attacks	Virtually impossible to brute force
Cloud providers	Often disabled by default	Required by many cloud providers (AWS, etc)

Guide: Switching SSH from Password to Key Authentication

Setup: Windows 11 PC is the **local machine**. nezuko (Ubuntu) is the **remote server**, normally accessed via Tailscale IP 100.76.26.45. All PowerShell steps run on Windows; all bash steps run on nezuko via SSH.

The padlock analogy: the PRIVATE key is the key itself - keep it safe, never share it. The PUBLIC key is the padlock - copy this onto nezuko. Only the matching key opens that padlock.

> **WARNING: KEEP A WORKING SSH SESSION OPEN AT ALL TIMES during this process. Tailscale (ssh scottp@100.76.26.45 or ssh scottp@nezuko) is your permanent fallback if anything goes wrong - never close every session until Step 14 confirms success.**

Steps 1-2: Identify local machine and open PowerShell

The Windows 11 PC is the LOCAL machine for this entire guide. Open PowerShell on Windows - NOT Command Prompt. Some of the later commands (Get-Service, the key-copy command, creating the config file) are PowerShell-only and will not work in classic CMD.

```
# Open PowerShell:
Win + X -> Terminal (or Windows PowerShell)
```

Step 3: Generate the SSH key pair

`ssh-keygen -t ed25519 -C "email"`

Generate a new public/private SSH key pair

What it does:
Creates two linked files - a private key and a public key - using the ed25519 algorithm. The private key proves your identity; the public key is placed on servers you want to access.

Flag	Meaning
-t ed25519	Type of key - ed25519 is the algorithm used
-C "comment"	A comment/label attached to the key - typically your email, purely informational

```
ssh-keygen -t ed25519 -C "scottpattison1974@googlemail.com"
```

Files created:

File	What it is	What to do with it
~/.ssh/id_ed25519	PRIVATE KEY - the key	NEVER share. Stays on Windows PC only
~/.ssh/id_ed25519.pub	PUBLIC KEY - the padlock	Copy this to nezuko

ADDED: Why ed25519 over other key types

Added because: the transcript states ed25519 is used but doesn't fully justify it - knowing WHY matters for understanding modern security practice.

ed25519 = Best - modern standard, fastest, most secure, shortest key length
RSA 4096 = Good alternative - older standard, still secure, but slower and longer
RSA 2048 = Outdated - considered too weak for new keys
DSA = Never use - deprecated, insecure

For any new key generated in 2026, ed25519 is the correct default choice.

Step 4: Respond to the prompts

Prompt	What to do
Enter file in which to save the key	Press Enter for default location
Enter passphrase (empty for no passphrase)	Press Enter for none, or type one for extra security
Enter same passphrase again	Press Enter again (or retype passphrase)

Step 5: Confirm the files were created

```
# Default save location on Windows:
C:\Users\scott\.ssh\

# Verify in PowerShell:
Get-ChildItem C:\Users\scott\.ssh\
# Should show:
# id_ed25519 (private key - no extension)
# id_ed25519.pub (public key - .pub extension)
```

Steps 6-7: Start ssh-agent and add the key

`ssh-agent / ssh-add`

Hold your private key in memory for the session

What they do:
ssh-agent is a background helper that holds your decrypted private key in memory so you don't have to re-enter a passphrase on every connection. ssh-add loads your key into the running agent.

```
# Step 6 - check and start ssh-agent (PowerShell):
Get-Service ssh-agent
# If status shows 'Stopped':
Get-Service ssh-agent | Set-Service -StartupType Automatic
Start-Service ssh-agent

# Step 7 - add your private key to the agent:
ssh-add ~/.ssh/id_ed25519
# If you set a passphrase in Step 4, it will be requested here
```

Step 8: Copy the public key to nezuko

Windows has no ssh-copy-id command (that's Linux-only), so PowerShell uses 'type' piped through ssh to achieve the same result. This will prompt for nezuko's password ONE LAST TIME.

```
type $env:USERPROFILE\.ssh\id_ed25519.pub | ssh scottp@100.76.26.45 "mkdir -p ~/.ssh &&
chmod 700 ~/.ssh && cat >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys"
```

Breaking the command down:

Part	What it does
type $env:USERPROFILE\.ssh\id_ed25519.pub	Reads the public key file content on Windows (type = PowerShell's cat)
| ssh scottp@100.76.26.45 "..."	Pipes that content to nezuko and runs the quoted commands there
mkdir -p ~/.ssh	Creates the .ssh folder on nezuko if it doesn't already exist
chmod 700 ~/.ssh	Sets correct permissions on the .ssh folder (owner only)
cat >> ~/.ssh/authorized_keys	Appends the piped public key onto nezuko's authorized_keys file
chmod 600 ~/.ssh/authorized_keys	Sets correct permissions on authorized_keys (owner read/write only)

Step 9: Test key authentication

```
ssh scottp@100.76.26.45
# If it connects WITHOUT asking for a password -> key auth is working
# If it still asks for a password -> do NOT proceed, recheck Step 8
```

Open two more sessions the same way and keep all three open. These connections stay alive even if a later step (restarting SSH) causes a problem - they are your safety net for Steps 11-12.

Step 10: Verify authorized_keys on nezuko

This step runs ON nezuko (in one of your open SSH sessions).

```
cat ~/.ssh/authorized_keys
# Should show your public key starting with:
# ssh-ed25519 AAAAC3... scottpattison1974@googlemail.com

ls -la ~/.ssh/
# Should show:
# drwx------ .ssh/ (700)
# -rw------- authorized_keys (600)

# If permissions are wrong, fix them:
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

Why permissions matter:

SSH is deliberately strict about security. If .ssh or authorized_keys have permissions that are too open (readable/writable by group or others), SSH will REFUSE to use the key and silently fall back to password authentication - with no obvious error message. The permissions must be exactly:

Item	Permission	Meaning
.ssh folder	700	Only the owner can access it (rwx------)
authorized_keys	600	Only the owner can read/write it (rw-------)

Step 11: Edit sshd_config on nezuko

`sudo vi /etc/ssh/sshd_config`

Edit the SSH server's configuration file

```
sudo vi /etc/ssh/sshd_config

# Use vi search to jump to each setting:
/PubkeyAuthentication
/AuthorizedKeysFile
/PasswordAuthentication

# Set these three lines exactly (remove leading # if present):
PubkeyAuthentication yes
AuthorizedKeysFile .ssh/authorized_keys
PasswordAuthentication no

# Save and exit:
Esc
:wq
```

WARNING: These lines may already exist but commented out with #, or may not exist at all (add them). Make sure there is only ONE of each line - duplicates can cause unpredictable behaviour.

Step 12: Restart SSH on nezuko

`sudo systemctl restart ssh`

Apply the new sshd_config by restarting the SSH service

```
# In Session 1:
sudo systemctl restart ssh

# Immediately switch to Session 2 and run any command:
pwd
# If Session 2 still responds -> everything worked
```

If Session 2 dropped - recovery order:

Step	Action
1	Try reconnecting: ssh scottp@100.76.26.45
2	If that fails: ssh scottp@nezuko (via Tailscale - different network path)
3	If still locked out: physical access to nezuko (keyboard/monitor)

```
# Re-enable password auth if needed (requires physical access):
sudo vi /etc/ssh/sshd_config
# Change: PasswordAuthentication no -> PasswordAuthentication yes
sudo systemctl restart ssh
```

Step 13: Create SSH config file on Windows

Back on the Windows PC - create a file at C:\Users\scott\.ssh\config (exactly named 'config', NO extension). This file lets you type 'ssh nezuko' instead of the full username and IP every time.

```
Host nezuko
HostName 100.76.26.45
User scottp
IdentityFile ~/.ssh/id_ed25519
```

Saving correctly from Notepad:

Step	Action
1	File -> Save As
2	Change 'Save as type' to All Files
3	Type filename exactly as: config
4	Save into C:\Users\scott\.ssh\

```
# Verify in PowerShell:
Get-ChildItem C:\Users\scott\.ssh\
# Should show: config, id_ed25519, id_ed25519.pub
# (config must have NO .txt extension)
```

Steps 14-15: Final test and verification

```
# Step 14 - close all sessions, open a fresh PowerShell window:
ssh nezuko
# Should connect instantly with NO password

# If it doesn't work, get verbose diagnostics:
ssh -v nezuko
```

Why 'ssh nezuko' works:
The Host entry 'nezuko' in your config file matches what you typed. PowerShell automatically substitutes HostName (100.76.26.45), User (scottp), and IdentityFile (your private key) - key auth handles the rest.

```
# Step 15 - on nezuko, confirm final config:
sudo grep PasswordAuthentication /etc/ssh/sshd_config
# Should return: PasswordAuthentication no

sudo grep PubkeyAuthentication /etc/ssh/sshd_config
# Should return: PubkeyAuthentication yes
```

Summary - Before and After

Before	After
ssh scottp@100.76.26.45 + password every time	ssh nezuko - instant, no password
Password sent over the network on every login	Private key never leaves the Windows PC
Vulnerable to brute force attacks	Virtually impossible to crack

GOLDEN RULES GOING FORWARD: Never delete id_ed25519 from the Windows PC - it is the only key that works. Never share the private key with anyone, for any reason. Never commit the private key to GitHub (the .pub file is safe to share but isn't needed publicly). ~/.ssh/config on Windows means 'ssh nezuko' now just works.

Quick Reference - All Commands

Command	What it does
ip a	Show all network interfaces and IP addresses
ip route	Show the routing table
ss -tuln	Show all listening TCP/UDP ports
hostname	Print current hostname
hostnamectl	Show full system info, or change hostname with set-hostname
sudo hostnamectl set-hostname NAME	Permanently change hostname
sudo nano /etc/hosts	Edit hostname mapping after a hostname change
ssh user@host	Connect to a remote server via SSH (port 22)
ssh-keygen -t ed25519 -C "email"	Generate a new SSH key pair
ssh-agent / ssh-add	Hold private key in memory for the session
cat ~/.ssh/authorized_keys	View keys authorised to log in as this user
chmod 700 ~/.ssh	Correct permission for .ssh folder
chmod 600 ~/.ssh/authorized_keys	Correct permission for authorized_keys file
sudo vi /etc/ssh/sshd_config	Edit SSH server configuration
sudo systemctl restart ssh	Apply new SSH configuration
ssh -v host	Verbose SSH connection - for troubleshooting

Linux+ Exam Tips: Know well-known ports cold - 22 (SSH), 53 (DNS), 80 (HTTP), 443 (HTTPS). Know ports are 0-65535 because they're a 16-bit value. Know the SSH connection sequence: connect on port 22 -> key exchange -> authentication -> encrypted channel. Know /etc/hosts must be updated manually after hostnamectl set-hostname. Know SSH key permissions: .ssh=700, authorized_keys=600 - wrong permissions silently fall back to password auth. Know IPv4 (32-bit, approx 4.3 billion) vs IPv6 (128-bit, approx 340 undecillion) and why IPv6 was created.